

MCP Server: Resources and Prompts

In the previous lecture, we learned how an MCP Server can expose **Tools** to an AI application.

A typical flow looked like this:

```
AI Application
  ↓
MCP Client
  ↓
MCP Server
  ↓
Tool
```

For example, an MCP Server may expose tools such as:

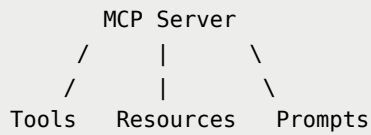
```
createTask()
getWeather()
sendEmail()
```

The LLM can inspect the available tools and decide when one of them should be invoked.

```
User
  ↓
LLM
  ↓
createTask(...)
  ↓
MCP Server
  ↓
Task created
```

Tools, however, are only one thing an MCP Server can expose.

An MCP Server can expose three major primitives:



A useful mental model is:

```
Tools      → Capabilities
Resources  → Context
Prompts    → Reusable AI workflows
```

More importantly, they differ in **who primarily controls them**:

```
Tools      → Model controlled
Resources  → Application controlled
Prompts    → User controlled
```

1. MCP Resources

Why do we need Resources?

Suppose an MCP Server contains information such as:

```
Coder Army Coding Guidelines

1. Use Java 21.
2. Prefer constructor injection.
3. Controllers should not contain business logic.
4. Services should contain business logic.
5. Use meaningful variable names.
```

We could technically expose this through a Tool:

```
@McpTool(...)
public String getCodingGuidelines() {
```

```
    return "...";  
}
```

This would work.

Tools are not limited to performing actions. They can also retrieve information.

The important question is therefore not:

| Does this return data?

The better question is:

| What is the intent of this capability?

Consider:

```
createTask()
```

This represents something the model can **call or execute**.

Now compare it with:

```
coding-guidelines.txt
```

This represents information that may be useful as **context**.

That gives us the deeper distinction:

```
Tool  
→ A capability the model can invoke  
  
Resource  
→ Context/data the application can make available
```

What is an MCP Resource?

An **MCP Resource** is data or content exposed by an MCP Server that a client can discover and read.

Examples include:

- Company policies
- README files
- Project documentation
- Git history
- Database records
- Application configuration
- API documentation
- Logs
- Knowledge-base articles
- Project architecture

A Resource does not automatically become part of the LLM context.

The flow is closer to:

```
graph TD; A[MCP Server] --> B[Exposes Resources]; B --> C[MCP Client / Host]; C --> D[Discovers Resources]; D --> E[Application]; E --> F[Controls which Resource is read]; F --> G[Relevant Resource]; G --> H[Can be supplied to the LLM as context];
```

MCP Server
↓
Exposes Resources
MCP Client / Host
↓
Discovers Resources
Application
↓
Controls which Resource is read
Relevant Resource
↓
Can be supplied to the LLM as context

This is why Resources are generally described as **application-controlled**.

Resource Discovery and Reading

MCP provides separate operations for discovering and reading Resources.

Discover available Resources

Conceptually:

```
resources/list
  ↓
"What Resources exist?"
```

Example:

```
CLIENT
|
|----- resources/list ----->
|
SERVER
|
| coding-guidelines
| URI: company://coding-guidelines
|
<-----
```

At this point, the client knows the Resource exists.

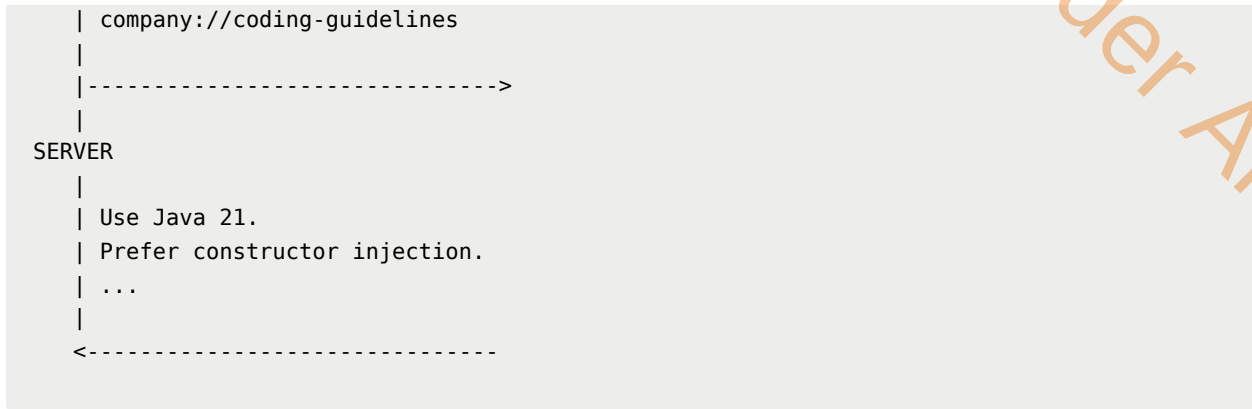
Read a Resource

When the content is required:

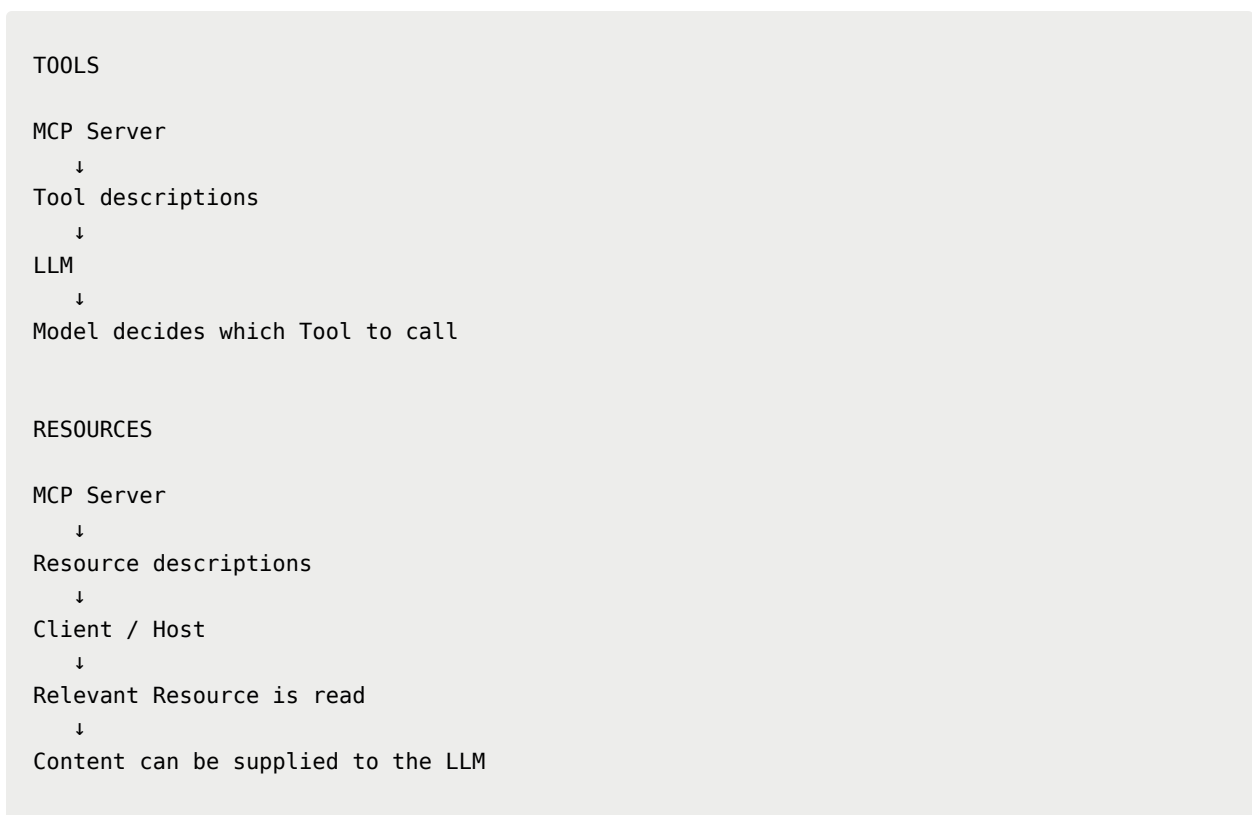
```
resources/read
  ↓
"Give me this Resource."
```

Example:

```
CLIENT
|
| resources/read
```



This gives us an important distinction:



Anatomy of an MCP Resource

A Resource usually contains metadata describing what it represents.

For our discussion, the important parts are:

Resource

- └ Name
- └ URI
- └ Description
- └ MIME Type
- └ Content

Name

The name identifies the Resource.

Example:

```
coding-guidelines
```

Other examples:

```
project-info
company-policy
developer-guide
database-schema
```

URI

A Resource needs an address through which the MCP client can uniquely identify it.

Example:

```
company://coding-guidelines
```

Other examples:

```
company://leave-policy
project://architecture
```

```
docs://spring-boot-guide  
config://application
```

The URI acts as the Resource's unique identifier.

URI does not necessarily mean URL

Consider:

```
company://coding-guidelines
```

This does not imply that the client performs:

```
HTTP GET company://coding-guidelines
```

The MCP client instead asks the MCP Server to read the Resource identified by that URI.

```
resources/read  
  
URI = company://coding-guidelines
```

The server then resolves the URI and returns the associated content.

Description

Descriptions help clients understand what a Resource represents.

```
Name:  
coding-guidelines  
  
Description:  
Coding standards used by our Java development team.
```

This becomes particularly useful when an MCP Server exposes many Resources.

MIME Type

The MIME type tells the client what kind of content the Resource contains.

Examples:

```
text/plain
application/json
text/html
image/png
```

For simple textual Resources, we can use:

```
text/plain
```

Content

The content is the actual information exposed by the Resource.

Example:

```
Use Java 21.

Prefer constructor injection.

Controllers should not contain business logic.

Services should contain application logic.
```

A complete Resource can therefore be understood as:

```
Name:
coding-guidelines

URI:
company://coding-guidelines

Description:
Coding guidelines followed by the development team.

MIME Type:
```

text/plain

Content:

Use Java 21.

Prefer constructor injection.

Keep business logic out of controllers.

A simple way to think about Resources is:

A Resource is an addressable piece of context along with metadata describing it.

Practical: Creating an MCP Resource

Suppose our existing MCP Server already contains task-management Tools.

We can add another Spring component for Resources:

```
@Component
public class TaskResources {

    @McpResource(
        uri = "task://guidelines",
        name = "task-guidelines",
        description = "Guidelines for managing and prioritizing tasks",
        mimeType = "text/plain"
    )
    public String taskGuidelines() {

        return """
            Task Management Guidelines:

            1. Deep work tasks should preferably be done before 12 PM.
            2. Meetings should preferably be scheduled after 2 PM.
            3. Urgent tasks should be completed on the same day.
            4. Keep a maximum of three high-priority tasks per day.
            5. Complete high-priority tasks before low-priority tasks.
            """;
    }
}
```

The server now exposes:

```
taskServer
```

```
| Tools  
|   | create_task  
|   | list_tasks  
|   | complete_task  
| Resources  
|   task://guidelines
```

When the application starts, Spring detects:

```
@McpResource
```

and registers the Resource with the MCP Server.

A client can discover it through:

```
resources/list
```

and read it through:

```
resources/read  
task://guidelines
```

Reading the Resource from the MCP Client

Unlike Tools, we do not expose the Resource to the model through:

```
defaultTools(...)
```

Instead, the application explicitly reads it.

A simplified example:

```
private String readTaskGuidelines() {

    var result = mcpClient.readResource(
        McpSchema.ReadResourceRequest
            .builder("task://guidelines")
            .build()
    );

    var content =
        (McpSchema.TextResourceContents)
            result.contents().getFirst();

    return content.text();
}
```

The flow becomes:

```
ChatService
  ↓
readResource("task://guidelines")
  ↓
MCP Server
  ↓
Task guidelines returned
  ↓
Application adds them to the prompt
  ↓
LLM
```

For example:

```
public String chat(String message) {

    String guidelines = readTaskGuidelines();

    return chatClient.prompt()
        .system("""
            You are a task management assistant.

            Use the following task management guidelines
            whenever relevant:

            %s
            """.formatted(guidelines))
        .user(message)
```

```
.call()  
.content();  
}
```

Now a question such as:

When should I schedule meetings?

can be answered using the MCP Resource:

Meetings should preferably be scheduled after 2 PM.

Static and Dynamic Resources

A Resource does not have to contain hardcoded text.

Our example:

```
task://guidelines
```

is a simple static Resource.

A Resource could also represent dynamic information:

```
user://123/profile  
  
database://customer/938  
  
project://coder-army/status
```

The Resource method could internally fetch data from:

- a database,
- another API,
- the filesystem,

- application state,
- or another service.

For example:

```
return database.findById(...);
```

Resource Templates

Suppose an application exposes user profiles:

```
user://1  
user://2  
user://3
```

Creating a separate Resource definition for every user would not be practical.

A URI template can instead be used:

```
user://{username}
```

Example:

```
@McpResource(  
    uri = "user://{username}",  
    name = "user-profile"  
)  
public String userProfile(String username) {  
  
    return "Profile of " + username;  
}
```

The server can now resolve:

```
user://aditya
```

or:

```
user://ravi
```

dynamically.

How Does the Application Decide Which Resource to Use?

Calling Resources **application-controlled** does not mean the application automatically understands natural language.

MCP does not prescribe how a host selects a Resource.

The application can use several strategies.

1. User Selection

A client may allow users to explicitly attach a Resource.

Attach Context

```
[ ] Coding Guidelines
[✓] Leave Policy
[ ] Architecture
[ ] Security Policy
```

Once the user selects:

Leave Policy

the application can perform:

```
resources/read
company://leave-policy
```

and add the content to the model context.

```
User selects Resource
      ↓
Application
      ↓
resources/read
      ↓
MCP Server
      ↓
Resource Content
      ↓
LLM Context
```

2. Application State

Sometimes the application already knows what context is relevant.

Consider an IDE.

The currently open file is:

```
OrderController.java
```

The user asks:

```
Explain this code.
```

The IDE already knows which file the user is referring to.

```
Current open file
      ↓
file:///project/src/OrderController.java
      ↓
resources/read
      ↓
File content
      ↓
LLM
```

No additional AI decision is required.

3. Retrieval or Search

If hundreds or thousands of Resources exist, the application can use retrieval.

Example Resource metadata:

```
Resource:
leave-policy

Description:
Company policies regarding leave,
holidays, maternity leave and time off.
```

User asks:

```
What is the maternity leave policy?
```

The application could perform:

```
User Query
  ↓
Embedding / Search
  ↓
Compare with Resource metadata
  ↓
Relevant Resource
  ↓
company://leave-policy
  ↓
resources/read
  ↓
LLM Context
```

This is conceptually similar to RAG.

4. LLM-Based Routing

The application can also use an LLM to select a Resource.

Suppose these Resources are available:

1. coding-guidelines
Java development standards
2. leave-policy
Employee leave and holiday policies
3. architecture
System architecture documentation

User asks:

What is our maternity leave policy?

A routing model could choose:

leave-policy

The application then performs:

resources/read(company://leave-policy)

The important distinction remains:

The MCP protocol did not make the Resource model-controlled. The application chose to use an LLM as part of its own routing strategy.

Resources vs Tools

A common confusion is:

If a Tool can retrieve information, why not expose everything as a Tool?

The answer depends on how we want the information to be used.

Use a **Tool** when the model should decide that something needs to be called.

```
searchDocumentation(query)
getWeather(city)
lookupOrder(id)
```

Use a **Resource** when the application should manage contextual/reference information.

```
company://coding-guidelines
file://README.md
project://architecture
```

A useful rule:

```
Should the MODEL decide to call it?
→ Tool

Should the APPLICATION provide it as context?
→ Resource
```

2. MCP Prompts

Resources solve the problem of reusable **context**.

There is another problem: reusable **ways of interacting with the AI**.

Suppose our task-management MCP Server wants to provide workflows such as:

```
Plan My Day
Prioritize My Tasks
Review My Workload
Create Weekly Plan
```

Consider:

Plan My Day

The workflow may require instructions such as:

Look at my pending tasks.

I have 4 hours available today.

Prioritize the tasks based on importance
and give me a concise ordered plan.

We could hardcode this inside every AI application.

```
String prompt = ""  
    Look at my pending tasks.  
  
    I have %s hours available today.  
  
    Prioritize the tasks based on importance  
    and give me a concise ordered plan.  
    "";
```

However, if multiple MCP clients consume the same server, each client would need to recreate the workflow.

MCP solves this through **Prompts**.

What is an MCP Prompt?

An MCP Prompt is a reusable message template exposed by an MCP Server.

Our server can now expose:

MCP SERVER

TOOLS

create_task

list_tasks

```
RESOURCES
task://
guidelines
```

```
PROMPTS
plan_day
```

Prompts are generally **user-controlled**.

A client may expose them through:

- Buttons
- Menus
- Slash commands
- Forms
- Other UI actions

Example:

Available Prompts

```
[ Plan My Day ]
[ Prioritize Tasks ]
[ Review Workload ]
```

The user explicitly chooses:

Plan My Day

The model does not independently decide to invoke `plan_day`.

Prompt vs LLM Response

An MCP Prompt does not generate the final AI answer.

It generates the **messages or instructions** that will later be sent to an LLM.

```
MCP Prompt
↓
Generates Messages
↓
MCP Client
↓
LLM
↓
Final Response
```

Therefore:

```
MCP Prompt ≠ LLM Response
```

The MCP Server defines a reusable interaction pattern.

The LLM still produces the actual answer.

Why Keep Prompts on the MCP Server?

A server can expose recommended workflows for using the capabilities and data it provides.

A task-management server might expose:

```
plan_day
prioritize_tasks
review_workload
```

A GitHub MCP Server might expose:

```
review_pull_request
summarize_issue
prepare_release_notes
```

A database MCP Server might expose:

```
analyze_sales  
explain_schema  
investigate_anomaly
```

An MCP Server can therefore communicate both:

```
"What can this system do?"
```

through Tools,

and:

```
"What are useful ways to work with this system?"
```

through Prompts.

Prompt Arguments

Prompts can accept arguments.

Suppose our Prompt is:

```
plan_day
```

Different users may have different amounts of available time:

```
2 hours  
4 hours  
8 hours
```

Instead of creating:

```
plan_day_2_hours  
plan_day_4_hours  
plan_day_8_hours
```

we create one Prompt:

```
plan_day(availableHours)
```

The client may display:

```
Plan My Day  
  
Available Hours:  
[ 4 ]  
  
[ Run ]
```

The client then requests:

```
Prompt:  
plan_day  
  
Arguments:  
availableHours = 4
```

The MCP Server can generate:

```
Plan my pending tasks for today.  
  
First check my current pending tasks.  
  
I have 4 hours available.  
  
Give me a concise ordered plan.
```

The argument is substituted into the reusable Prompt template.

Prompt Discovery

MCP provides two important operations conceptually:

```
prompts/list
prompts/get
```

Discover Prompts

```
CLIENT
|
|----- prompts/list ----->
|
SERVER
|
| plan_day
| availableHours required
|
|<-----
```

The client can then display:

```
Plan My Day
```

to the user.

Get a Prompt

After the user chooses it:

```
CLIENT
|
| prompts/get
|
| name = plan_day
| availableHours = 4
|
|----->
|
SERVER
```

```
|
| Prompt Messages
|
|----->
```

Practical: Creating an MCP Prompt

Our server can contain another component:

```
taskServer
├─ TaskTools.java
├─ TaskResources.java
└─ TaskPrompts.java
```

Example:

```
@Component
public class TaskPrompts {

    @McpPrompt(
        name = "plan_day",
        description = "Plan pending tasks for the available time"
    )
    public McpSchema.GetPromptResult planDay(

        @McpArg(
            name = "availableHours",
            description = "Number of hours available today",
            required = true
        )
        String availableHours
    ) {

        String message = ""
            Help me plan my pending tasks for today.

            First check my current pending tasks
            using the available task tools.

            I have %s hours available today.

            Give me a concise and ordered plan.
            """.formatted(availableHours);
```

Coder Army

```
var promptMessage =
    McpSchema.PromptMessage.builder(
        McpSchema.Role.USER,
        McpSchema.TextContent.builder(message).build()
    ).build();

return McpSchema.GetPromptResult
    .builder(List.of(promptMessage))
    .description("Plan pending tasks for the day")
    .build();
}
```

The Prompt accepts:

```
availableHours
```

and produces a message that will later be given to the LLM.

Prompt and Tool Working Together

Notice this part of the Prompt:

```
First check my current pending tasks
using the available task tools.
```

The Prompt does not execute:

```
list_tasks
```

It only instructs the LLM.

The LLM receives the Prompt and sees that `list_tasks` is one of its available Tools.

It can then decide:

```
I need to know the current tasks.
```

and invoke:

```
list_tasks
```

The flow becomes:

```
User selects Prompt
  ↓
Prompt guides LLM
  ↓
LLM decides a Tool is needed
  ↓
Tool is executed
```

This demonstrates how different MCP primitives can work together.

Reading a Prompt from the MCP Client

The client can request the Prompt using the same MCP connection.

Example:

```
public String planDay(String hours) {

    var result = mcpClient.getPrompt(
        McpSchema.GetPromptRequest
            .builder("plan_day")
            .arguments(
                Map.of("availableHours", hours)
            )
            .build()
    );

    var content =
        (McpSchema.TextContent)
            result.messages()
                .getFirst()
                .content();

    String generatedPrompt = content.text();

    return chatClient.prompt()
        .user(generatedPrompt)
```

```
.call()  
.content();  
}
```

The two operations should not be confused:

```
mcpClient.getPrompt(...)  
    ↓  
Communication with MCP Server  
  
chatClient.prompt(...)  
    ↓  
Communication with LLM
```

`getPrompt()` only retrieves the reusable Prompt messages.

The LLM call happens afterward.

Simulating User-Controlled Prompts with Postman

A real MCP client may display:

```
Prompts  
  
[ Plan My Day ]
```

In a simple demo application, an endpoint can simulate that user selection.

```
@GetMapping("/plan")  
public String plan(  
    @RequestParam("hours") String hours) {  
  
    return chatService.planDay(hours);  
}
```

Calling:

GET /plan?hours=4

means that the user has effectively selected:

Plan My Day

and provided:

availableHours = 4

Bringing Tools, Resources and Prompts Together

Our final task-management MCP Server can expose:

```
taskServer
├── Tools
│   ├── create_task
│   ├── list_tasks
│   └── complete_task
├── Resources
│   └── task://guidelines
└── Prompts
    └── plan_day
```

Now consider:

GET /plan?hours=4

Step 1 — User Controls the Prompt

The user chooses:

```
plan_day
```

with:

```
availableHours = 4
```

The MCP Server generates the reusable Prompt.

```
USER
↓
selects Plan My Day
↓
PROMPT
```

Step 2 — Application Controls the Resource

The application reads:

```
task://guidelines
```

and places the guidelines into the LLM context.

```
APPLICATION
↓
resources/read
↓
task://guidelines
↓
Context
```

Step 3 — Model Controls the Tool

The Prompt asks the model to inspect pending tasks.

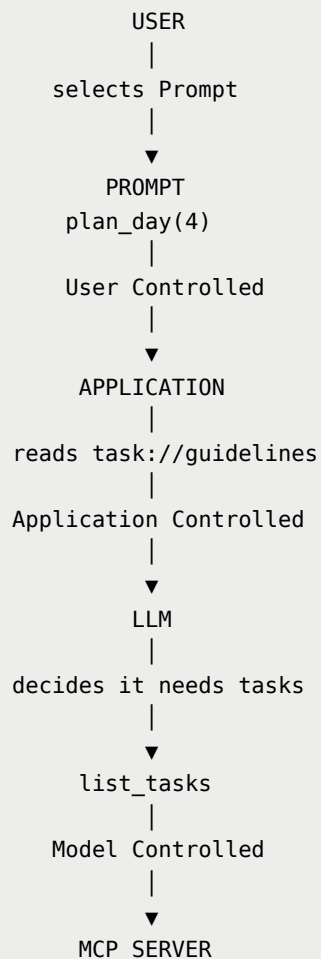
The LLM sees:

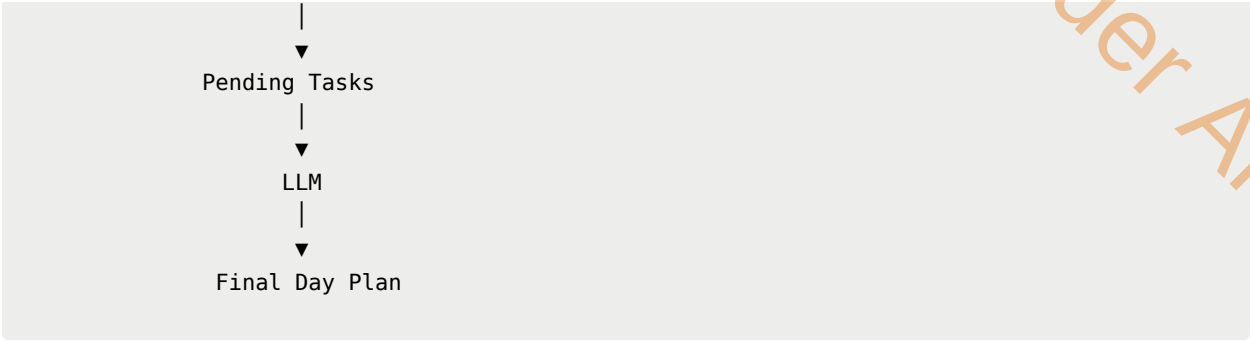
```
list_tasks
```

among its available Tools and decides to call it.

```
LLM
↓
list_tasks
↓
MCP Server
↓
Pending Tasks
```

The complete flow becomes:



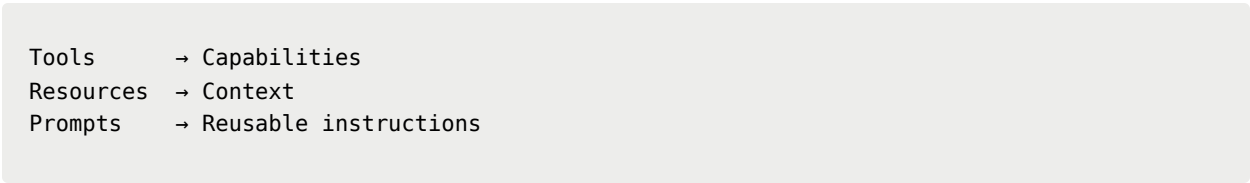


Tools vs Resources vs Prompts

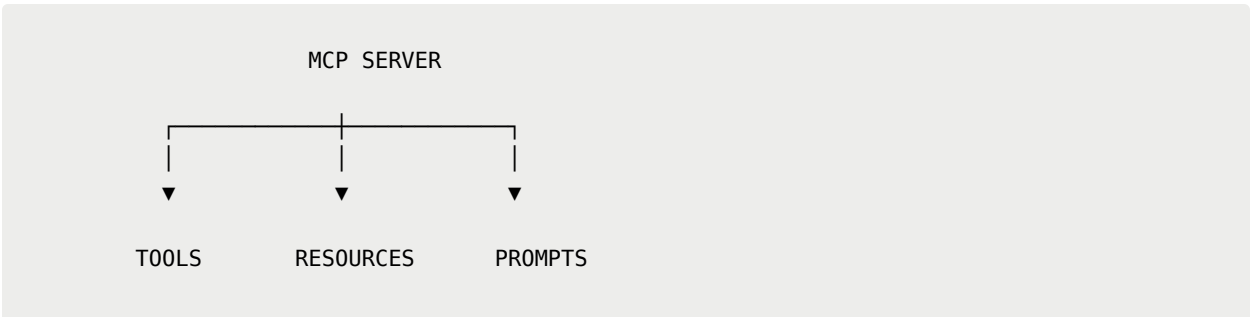
MCP Primitive	Main Purpose	Primarily Controlled By	Example
Tool	Expose callable capabilities	Model	<code>list_tasks</code>
Resource	Expose context or reference data	Application	<code>task://guidelines</code>
Prompt	Expose reusable interaction templates	User	<code>plan_day</code>

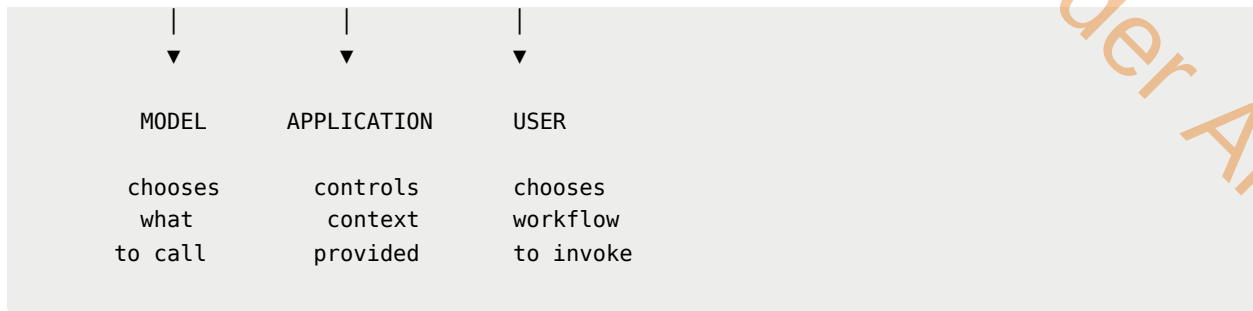
Final Mental Model

A simplified explanation is:



But the stronger distinction is based on **control**.





A useful set of questions is:

TOOLS

"What should the MODEL be able to call?"

RESOURCES

"What context should the APPLICATION be able to provide?"

PROMPTS

"What reusable workflows should the USER be able to choose?"

Key Takeaways

- An MCP Server can expose **Tools, Resources and Prompts**.
- Tools provide callable capabilities to the model.
- Tools can both perform actions and retrieve information.
- Resources expose contextual or reference data.
- Resources are identified using URIs.
- Resources are discovered and read separately.
- A Resource does not automatically become part of the LLM context.

- The host application controls when Resources are read and supplied to the model.
- Resource selection can be based on user selection, application state, retrieval, rules or LLM-based routing.
- Prompts expose reusable ways of interacting with an AI system.
- Prompts can accept arguments such as `availableHours`.
- A Prompt generates messages; it does not generate the final LLM response.
- Prompts are typically surfaced for explicit user selection.
- Tools, Resources and Prompts can work together in the same workflow.

Tools → Model Controlled
Resources → Application Controlled
Prompts → User Controlled

This completes the core server-side MCP model:

